Laboratoire — Développement d'un mécanisme de mise à jour automatique

1. Mise en situation

Les applications doivent régulièrement être mises à jour afin de corriger des bogues ou d'ajouter des fonctionnalités. Des outils comme ClickOnce permettent d'automatiser une partie de ce processus.

Dans le laboratoire formatif précédent, vous avez utilisé ClickOnce pour publier et mettre à jour une application. Vous devez maintenant développer votre propre mécanisme de mise à jour afin de comprendre les principales opérations réalisées par ce type d'outil.

Vous travaillerez à partir d'une application WPF existante, GestionNotes. Vous développerez un système permettant à l'application de détecter une nouvelle version, de récupérer les fichiers nécessaires et d'appliquer la mise à jour automatiquement.

Les fichiers distribués seront hébergés dans un dossier réseau partagé que vous devrez configurer vous-mêmes sur votre machine virtuelle Windows. Votre poste de travail et votre machine virtuelle se trouvent sur le même VLAN.

Votre solution devra également comporter un programme auxiliaire permettant de remplacer les fichiers de l'application après sa fermeture.

Important : vous ne devez pas utiliser ClickOnce, MSIX ou un autre outil de déploiement existant pour réaliser les mises à jour à votre place. Le mécanisme doit être programmé par vos soins.

2. Objectifs d'apprentissage

À la fin de ce laboratoire, vous serez en mesure de :

- Configurer un dossier réseau partagé sous Windows.
- Comprendre la différence entre les permissions de partage et les permissions NTFS.
- Comparer des numéros de version en C#.
- Lire et désérialiser un fichier JSON.
- Vérifier la disponibilité d'une nouvelle version.
- Copier des fichiers depuis un partage réseau vers un poste client.
- Utiliser les mécanismes de gestion des processus en C#.
- Coordonner la fermeture, le remplacement et le redémarrage d'une application.

- Gérer les erreurs courantes d'un mécanisme de mise à jour.
- Versionner et documenter une solution avec Git.

3. Matériel fourni

Vous recevrez les éléments suivants :

- **GestionNotes v1.0** : application WPF fonctionnelle comportant un bogue connu dans le calcul de la moyenne.
- **GestionNotes v1.1** : version corrigée du bogue.
- **GestionNotes v2.0** : version avec un meilleur visuel.
- Les projets Visual Studio nécessaires pour démarrer le développement.
- Une machine virtuelle Windows accessible depuis votre poste de travail.

Vous devez conserver le fonctionnement de l'application existante et y intégrer le mécanisme de mise à jour demandé.

Les deux versions fournies vous permettront de vérifier que le remplacement des fichiers produit réellement une nouvelle version de l'application.

4. Configuration du dépôt réseau

Avant de programmer le mécanisme de mise à jour, vous devez configurer un dossier réseau partagé sur votre machine virtuelle Windows.

Ce dossier servira de dépôt central pour les nouvelles versions de GestionNotes.

A.1 — Création du dossier

Sur votre machine virtuelle :

1. Créez un dossier nommé GestionNotes-Depot.
2. Configurez le partage réseau de ce dossier.
3. Déterminez les permissions de partage et les permissions NTFS nécessaires pour accéder aux fichiers.
4. Assurez-vous que le partage est accessible depuis votre poste de travail.

Le partage devra être accessible à l'aide d'un chemin UNC de la forme suivante :

\\NOM-DE-LA-VM\GestionNotes-Depot

Remplacez NOM-DE-LA-VM par le nom réel de votre machine virtuelle.

Vous ne devez pas utiliser un lecteur réseau mappé, comme Z:\GestionNotes-Depot, pour accéder au dépôt depuis votre application.

A.2 — Validation de l'accès

Depuis votre poste de travail :

1. Ouvrez l'Explorateur de fichiers.
2. Accédez au partage en utilisant son chemin UNC.
3. Créez ou copiez un fichier de test dans le dossier afin de confirmer que l'accès fonctionne.
4. Vérifiez que les fichiers du partage peuvent être consultés depuis le poste client.

Si l'accès échoue, vous devez diagnostiquer le problème avant de poursuivre. Vérifiez notamment la connectivité réseau, le nom de la machine virtuelle, le pare-feu Windows, le service de partage de fichiers et les permissions.

A.3 — Organisation du dépôt

Le dépôt devra contenir le manifeste JSON décrivant la version disponible et l'archive ZIP correspondante.

Exemple :

GestionNotes-Depot\

version.json

GestionNotes-1.1.zip

Vous devrez être en mesure de publier une nouvelle version en remplaçant les fichiers du dépôt, sans modifier le code source ni recompiler l'application cliente.

Inscrivez le chemin UNC de votre partage dans votre README.md.

5. Vérification des versions

Vous devez programmer le mécanisme qui permet à GestionNotes de déterminer si une nouvelle version est disponible.

Au démarrage de l'application, votre solution devra :

1. Accéder au fichier version.json dans le dossier réseau.
2. Lire et désérialiser le contenu du fichier.
3. Récupérer le numéro de la version disponible.
4. Comparer cette version à celle de l'application installée.
5. Déterminer si une mise à jour est nécessaire.

Le manifeste doit contenir au minimum les propriétés suivantes :

- version : numéro de la version disponible.

- fichier : nom de l'archive ZIP contenant cette version.

Exemple de contenu :

```
{
  "version": "1.1",
  "fichier": "GestionNotes-1.1.zip"
}
```

Vous pouvez ajouter d'autres propriétés si elles sont utiles à votre solution.

Contraintes :

- Utilisez System.Version pour comparer les versions.
- Ne comparez pas les numéros de version comme de simples chaînes de caractères.
- Si aucune nouvelle version n'est disponible, l'application doit continuer normalement.
- Si le partage est inaccessible ou si le manifeste est invalide, l'application doit afficher un message approprié et demeurer utilisable.

Par exemple, la version 1.10 doit être reconnue comme plus récente que la version 1.2.

6. Téléchargement de la mise à jour

Lorsqu'une mise à jour est disponible, l'application doit informer l'utilisateur et lui permettre de l'accepter ou de la refuser.

Si l'utilisateur accepte, votre solution doit :

1. Vérifier que l'archive ZIP existe dans le dépôt réseau.
2. Copier l'archive dans un répertoire temporaire local.
3. Vérifier que la copie s'est terminée correctement.
4. Préparer les informations nécessaires pour appliquer la mise à jour.

La mise à jour doit utiliser la copie locale de l'archive. Le processus de remplacement ne doit donc pas dépendre de la disponibilité du partage réseau.

Si l'utilisateur refuse la mise à jour, l'application doit continuer à fonctionner normalement.

Si le fichier ZIP est absent ou si la copie échoue, vous devez afficher un message d'erreur et ne pas fermer l'application pour tenter d'appliquer une mise à jour incomplète.

7. Développement du programme auxiliaire

Vous devez développer un deuxième programme nommé Updater.exe.

Ce programme sera responsable de l'application de la mise à jour. Il est nécessaire parce qu'une application ne devrait pas tenter de remplacer ses propres fichiers exécutables pendant qu'elle est en cours d'exécution.

Le programme auxiliaire devra :

1. Recevoir les informations nécessaires pour appliquer la mise à jour, notamment le chemin de l'archive locale et celui du répertoire d'installation.
2. Attendre que le processus GestionNotes soit complètement terminé.
3. Extraire l'archive ZIP dans un répertoire temporaire.
4. Remplacer les fichiers de l'ancienne version par ceux de la nouvelle version.
5. Démarrer la nouvelle version de GestionNotes.
6. Terminer son exécution lorsque le redémarrage a été lancé.

Le programme principal doit lancer Updater.exe, puis se fermer proprement.

Vous devez utiliser les mécanismes de gestion des processus de C# pour coordonner les deux programmes.

Attention : le programme auxiliaire ne doit pas commencer à remplacer les fichiers avant la fermeture complète de GestionNotes.

8. Gestion des erreurs

Votre solution doit gérer correctement les situations suivantes :

- Le dossier réseau est inaccessible.
- Le fichier version.json est absent.
- Le manifeste JSON est invalide.
- Aucune nouvelle version n'est disponible.
- L'archive ZIP est absente du dépôt.
- La copie de l'archive échoue.
- L'archive ZIP ne peut pas être extraite.
- L'utilisateur refuse la mise à jour.

Lorsqu'une erreur survient, l'application doit informer l'utilisateur et éviter de laisser l'installation inutilisable.

Vous devez notamment éviter de fermer l'application principale si la mise à jour ne peut pas être préparée correctement.

9. Publication et validation

Une fois votre mécanisme développé, vous devez déployer la version 1.0 de GestionNotes sur votre poste client et placer la version 1.1 dans le dépôt réseau.

Vous devez démontrer les scénarios suivants.

Scénario 1 — Mise à jour réussie

- L'application installée est en version 1.0.
- Le dépôt réseau annonce la version 1.1.
- L'application détecte la mise à jour.
- Vous acceptez la mise à jour.
- L'application se ferme, les fichiers sont remplacés et l'application redémarre en version 1.1.

Scénario 2 — Aucune mise à jour nécessaire

- L'application installée est en version 1.1.
- Le dépôt annonce également la version 1.1.
- Aucune mise à jour n'est appliquée.
- L'application continue de fonctionner normalement.

Scénario 3 — Partage réseau inaccessible

- Rendez le dépôt réseau temporairement inaccessible ou utilisez un chemin invalide pour le test.
- Démarrez GestionNotes.
- Vérifiez que l'application affiche un message approprié et demeure utilisable.

Rétablissez ensuite l'accès au dépôt.

Scénario 4 — Refus de la mise à jour

- Rendez une version plus récente disponible.
- Refusez la mise à jour lorsque l'application vous le propose.
- Vérifiez que l'application continue de fonctionner avec la version déjà installée.

Scénario 5 — Archive absente

- Modifiez temporairement le manifeste pour qu'il désigne un fichier ZIP inexistant.
- Vérifiez que votre application détecte l'erreur et n'essaie pas d'appliquer une mise à jour incomplète.

Rétablissez ensuite le manifeste et les fichiers du dépôt.

10. Contraintes techniques

Votre solution doit respecter les contraintes suivantes :

- Le code doit être écrit en C#.
- L'application principale doit conserver son architecture WPF et MVVM.
- Le manifeste doit être au format JSON ou XML.
- Le dépôt des mises à jour doit être un dossier réseau partagé sous Windows.
- L'accès au dépôt doit utiliser un chemin UNC.
- Les mises à jour ne doivent pas utiliser HTTP.
- L'archive doit être copiée localement avant l'application de la mise à jour.
- Le remplacement des fichiers doit être réalisé par Updater.exe après la fermeture de GestionNotes.
- Le code doit gérer les erreurs courantes sans rendre l'application inutilisable.
- La solution doit être versionnée dans un dépôt Git.

Vous pouvez utiliser les fonctionnalités intégrées à .NET pour la sérialisation JSON, la manipulation des fichiers, l'extraction des archives ZIP et la gestion des processus.

Vous ne devez pas utiliser ClickOnce, MSIX ou un autre outil de déploiement pour effectuer automatiquement les mises à jour à votre place.

11. Remise

Vous devez remettre un lien vers votre dépôt Git contenant :

- Le code source de GestionNotes.
- Le code source de Updater.
- Les fichiers nécessaires pour construire le manifeste JSON.
- Un fichier README.md présentant les étapes de configuration, de publication et de validation.

Le README doit expliquer :

1. Quel chemin UNC est utilisé par votre application.
2. Comment publier une nouvelle version.
3. Comment fonctionne votre mécanisme de mise à jour.

Ajoutez des captures d'écran montrant le partage réseau accessible, la version avant la mise à jour, la détection de la nouvelle version et la version après la mise à jour.

Le dépôt doit permettre à une autre personne de comprendre votre solution et de reproduire les tests.

12. Barème d'évaluation

Critère	**Points**
Configuration et validation du partage réseau	2
Lecture du manifeste et comparaison des versions	4
Copie locale de l'archive et gestion des erreurs	4
Fonctionnement de Updater.exe et remplacement des fichiers	5
Redémarrage de l'application et validation des scénarios	3
Qualité du code, documentation et remise Git	2
Total	**20**

13. Résultat attendu

À la fin du laboratoire, vous devez disposer d'un mécanisme fonctionnel permettant de publier une nouvelle version de GestionNotes dans un dossier réseau partagé et de mettre à jour une installation existante.

Votre solution doit démontrer votre compréhension des étapes essentielles d'un mécanisme de mise à jour automatique : vérification de la version, récupération des fichiers, fermeture de l'application, remplacement des fichiers et redémarrage.

Vous devez également être en mesure d'expliquer pourquoi le remplacement des fichiers est effectué par un programme auxiliaire et pourquoi l'archive est copiée localement avant l'application de la mise à jour.